page 760 as the equation $T_P = T_1/P + T_\infty$, which we can use as an approximation to the running time on $P$ processors. Then indeed we have $T_{32} = 2048/32 + 1 = 65$. With the optimization, the work becomes $T_1' = 1024$ seconds, and the span becomes $T_\infty' = 8$ seconds. Our approximation gives $T_{32}' = 1024/32 + 8 = 40$.

The relative speeds of the two versions switch when we estimate their running times on 512 processors, however. The first version has a running time of $T_{512} = 2048/512+1 = 5$ seconds, and the second version runs in $T_{512}' = 1024/512+8 = 10$ seconds. The optimization that speeds up the program on 32 processors makes the program run for twice as long on 512 processors! The optimized version's span of 8, which is not the dominant term in the running time on 32 processors, becomes the dominant term on 512 processors, nullifying the advantage from using more processors. The optimization does not scale up.

The moral of the story is that work/span analysis, and measurements of work and span, can be superior to measured running times alone in extrapolating an algorithm's scalability.

## Exercises

### *26.1-1*

What does a trace for the execution of a serial algorithm look like?

### *26.1-2*

Suppose that line 4 of P-FIB spawns P-FIB$(n-2)$, rather than calling it as is done in the pseudocode. How would the trace of P-FIB$(4)$ in Figure 26.2 change? What is the impact on the asymptotic work, span, and parallelism?

### *26.1-3*

Draw the trace that results from executing P-FIB$(5)$. Assuming that each strand in the computation takes unit time, what are the work, span, and parallelism of the computation? Show how to schedule the trace on 3 processors using greedy scheduling by labeling each strand with the time step in which it is executed.

### *26.1-4*

Prove that a greedy scheduler achieves the following time bound, which is slightly stronger than the bound proved in Theorem 26.1:

$$T_P \le \frac{T_1 - T_\infty}{P} + T_\infty \,. \tag{26.5}$$

### *26.1-5*

Construct a trace for which one execution by a greedy scheduler can take nearly twice the time of another execution by a greedy scheduler on the same number of processors. Describe how the two executions would proceed.

***26.1-6***

Professor Karan measures her deterministic task-parallel algorithm on 4, 10, and 64 processors of an ideal parallel computer using a greedy scheduler. She claims that the three runs yielded $T_4 = 80$ seconds, $T_{10} = 42$ seconds, and $T_{64} = 10$ seconds. Argue that the professor is either lying or incompetent. (*Hint:* Use the work law (26.2), the span law (26.3), and inequality (26.5) from Exercise 26.1-4.)

***26.1-7***

Give a parallel algorithm to multiply an $n \times n$ matrix by an $n$-vector that achieves $\Theta(n^2/\lg n)$ parallelism while maintaining $\Theta(n^2)$ work.

***26.1-8***

Analyze the work, span, and parallelism of the procedure P-TRANSPOSE, which transposes an $n \times n$ matrix $A$ in place.

```
P-TRANSPOSE(A, n)
1  parallel for j = 2 to n
2      parallel for i = 1 to j − 1
3          exchange a_ij with a_ji
```

***26.1-9***

Suppose that instead of a **parallel for** loop in line 2, the P-TRANSPOSE procedure in Exercise 26.1-8 had an ordinary **for** loop. Analyze the work, span, and parallelism of the resulting algorithm.

***26.1-10***

For what number of processors do the two versions of the chess program run equally fast, assuming that $T_P = T_1/P + T_\infty$?

## 26.2 Parallel matrix multiplication

In this section, we'll explore how to parallelize the three matrix-multiplication algorithms from Sections 4.1 and 4.2. We'll see that each algorithm can be parallelized in a straightforward fashion using either parallel loops or recursive spawning. We'll analyze them using work/span analysis, and we'll see that each parallel algorithm attains the same performance on one processor as its corresponding serial algorithm, while scaling up to large numbers of processors.

### A parallel algorithm for matrix multiplication using parallel loops

The first algorithm we'll study is P-MATRIX-MULTIPLY, which simply parallelizes the two outer loops in the procedure MATRIX-MULTIPLY on page 81.

```
P-MATRIX-MULTIPLY(A, B, C, n)
1  parallel for i = 1 to n                  // compute entries in each of n rows
2      parallel for j = 1 to n              // compute n entries in row i
3          for k = 1 to n
4              c_ij = c_ij + a_ik · b_kj    // add in another term of equation (4.1)
```

Let's analyze P-MATRIX-MULTIPLY. Since the serial projection of the algorithm is just MATRIX-MULTIPLY, the work is the same as the running time of MATRIX-MULTIPLY: $T_1(n) = \Theta(n^3)$. The span is $T_\infty(n) = \Theta(n)$, because it follows a path down the tree of recursion for the **parallel for** loop starting in line 1, then down the tree of recursion for the **parallel for** loop starting in line 2, and then executes all $n$ iterations of the ordinary **for** loop starting in line 3, resulting in a total span of $\Theta(\lg n) + \Theta(\lg n) + \Theta(n) = \Theta(n)$. Thus the parallelism is $\Theta(n^3)/\Theta(n) = \Theta(n^2)$. (Exercise 26.2-3 asks you to parallelize the inner loop to obtain a parallelism of $\Theta(n^3/\lg n)$, which you cannot do straightforwardly using **parallel for**, because you would create races.)

### A parallel divide-and-conquer algorithm for matrix multiplication

Section 4.1 shows how to multiply $n \times n$ matrices serially in $\Theta(n^3)$ time using a divide-and-conquer strategy. Let's see how to parallelize that algorithm using recursive spawning instead of calls.

The serial MATRIX-MULTIPLY-RECURSIVE procedure on page 83 takes as input three $n \times n$ matrices $A$, $B$, and $C$ and performs the matrix calculation $C = C + A \cdot B$ by recursively performing eight multiplications of $n/2 \times n/2$ submatrices of $A$ and $B$. The P-MATRIX-MULTIPLY-RECURSIVE procedure on the following page implements the same divide-and-conquer strategy, but it uses spawning to perform the eight multiplications in parallel. To avoid determinacy races in updating the elements of $C$, it creates a temporary matrix $D$ to store four of the submatrix products. At the end, it adds $C$ and $D$ together to produce the final result. (Problem 26-2 asks you to eliminate the temporary matrix $D$ at the expense of some parallelism.)

Lines 2–3 of P-MATRIX-MULTIPLY-RECURSIVE handle the base case of multiplying $1 \times 1$ matrices. The remainder of the procedure deals with the recursive case. Line 4 allocates a temporary matrix $D$, and lines 5–7 zero it. Line 8 partitions each of the four matrices $A$, $B$, $C$, and $D$ into $n/2 \times n/2$ submatrices. (As

```
P-MATRIX-MULTIPLY-RECURSIVE(A, B, C, n)
 1  if n == 1                          // just one element in each matrix?
 2      c_11 = c_11 + a_11 · b_11
 3      return
 4  let D be a new n × n matrix        // temporary matrix
 5  parallel for i = 1 to n            // set D = 0
 6      parallel for j = 1 to n
 7          d_ij = 0
 8  partition A, B, C, and D into n/2 × n/2 submatrices
        A_11, A_12, A_21, A_22; B_11, B_12, B_21, B_22; C_11, C_12, C_21, C_22;
        and D_11, D_12, D_21, D_22; respectively
 9  spawn P-MATRIX-MULTIPLY-RECURSIVE(A_11, B_11, C_11, n/2)
10  spawn P-MATRIX-MULTIPLY-RECURSIVE(A_11, B_12, C_12, n/2)
11  spawn P-MATRIX-MULTIPLY-RECURSIVE(A_21, B_11, C_21, n/2)
12  spawn P-MATRIX-MULTIPLY-RECURSIVE(A_21, B_12, C_22, n/2)
13  spawn P-MATRIX-MULTIPLY-RECURSIVE(A_12, B_21, D_11, n/2)
14  spawn P-MATRIX-MULTIPLY-RECURSIVE(A_12, B_22, D_12, n/2)
15  spawn P-MATRIX-MULTIPLY-RECURSIVE(A_22, B_21, D_21, n/2)
16  spawn P-MATRIX-MULTIPLY-RECURSIVE(A_22, B_22, D_22, n/2)
17  sync                               // wait for spawned submatrix products
18  parallel for i = 1 to n            // update C = C + D
19      parallel for j = 1 to n
20          c_ij = c_ij + d_ij
```

with MATRIX-MULTIPLY-RECURSIVE on page 83, we're glossing over the subtle issue of how to use index calculations to represent submatrix sections of a matrix.) The spawned recursive call in line 9 sets $C_{11} = C_{11} + A_{11} \cdot B_{11}$, so that $C_{11}$ accumulates the first of the two terms in equation (4.5) on page 82. Similarly, lines 10–12 cause each of $C_{12}$, $C_{21}$, and $C_{22}$ in parallel to accumulate the first of the two terms in equations (4.6)–(4.8), respectively. Line 13 sets the submatrix $D_{11}$ to the submatrix product $A_{12} \cdot B_{21}$, so that $D_{11}$ equals the second of the two terms in equation (4.5). Lines 14–16 set each of $D_{12}$, $D_{21}$, and $D_{22}$ in parallel to the second of the two terms in equations (4.6)–(4.8), respectively. The **sync** statement in line 17 ensures that all the spawned submatrix products in lines 9–16 have been computed, after which the doubly nested **parallel for** loops in lines 18–20 add the elements of $D$ to the corresponding elements of $C$.

Let's analyze the P-MATRIX-MULTIPLY-RECURSIVE procedure. We start by analyzing the work $M_1(n)$, echoing the serial running-time analysis of its progenitor MATRIX-MULTIPLY-RECURSIVE. The recursive case allocates and zeros the

temporary matrix $D$ in $\Theta(n^2)$ time, partitions in $\Theta(1)$ time, performs eight recursive multiplications of $n/2 \times n/2$ matrices, and finishes up with the $\Theta(n^2)$ work from adding two $n \times n$ matrices. Thus the work outside the spawned recursive calls is $\Theta(n^2)$, and the recurrence for the work $M_1(n)$ becomes

$$\begin{aligned} M_1(n) &= 8M_1(n/2) + \Theta(n^2) \\ &= \Theta(n^3) \end{aligned}$$

by case 1 of the master theorem (Theorem 4.1). Not surprisingly, the work of this parallel algorithm is asymptotically the same as the running time of the procedure MATRIX-MULTIPLY on page 81, with its triply nested loops.

Let's determine the span $M_\infty(n)$ of P-MATRIX-MULTIPLY-RECURSIVE. Because the eight parallel recursive spawns all execute on matrices of the same size, the maximum span for any recursive spawn is just the span of a single one of them, or $M_\infty(n/2)$. The span for the doubly nested **parallel for** loops in lines 5–7 is $\Theta(\lg n)$ because each loop control adds $\Theta(\lg n)$ to the constant span of line 7. Similarly, the doubly nested **parallel for** loops in lines 18–20 add another $\Theta(\lg n)$. Matrix partitioning by index calculation has $\Theta(1)$ span, which is dominated by the $\Theta(\lg n)$ span of the nested loops. We obtain the recurrence

$$M_\infty(n) = M_\infty(n/2) + \Theta(\lg n) \,. \tag{26.6}$$

Since this recurrence falls under case 2 of the master theorem with $k = 1$, the solution is $M_\infty(n) = \Theta(\lg^2 n)$.

The parallelism of P-MATRIX-MULTIPLY-RECURSIVE is $M_1(n)/M_\infty(n) = \Theta(n^3/\lg^2 n)$, which is huge. (Problem 26-2 asks you to simplify this parallel algorithm at the expense of just a little less parallelism.)

### Parallelizing Strassen's method

To parallelize Strassen's algorithm, we can follow the same general outline as on pages 86–87, but use spawning. You may find it helpful to compare each step below with the corresponding step there. We'll analyze costs as we go along to develop recurrences $T_1(n)$ and $T_\infty(n)$ for the overall work and span, respectively.

1. If $n = 1$, the matrices each contain a single element. Perform a single scalar multiplication and a single scalar addition, and return. Otherwise, partition the input matrices $A$ and $B$ and output matrix $C$ into $n/2 \times n/2$ submatrices, as in equation (4.2) on page 82. This step takes $\Theta(1)$ work and $\Theta(1)$ span by index calculation.
2. Create $n/2 \times n/2$ matrices $S_1, S_2, \ldots, S_{10}$, each of which is the sum or difference of two submatrices from step 1. Create and zero the entries of seven $n/2 \times n/2$ matrices $P_1, P_2, \ldots, P_7$ to hold seven $n/2 \times n/2$ matrix products. All

17 matrices can be created, and the $P_i$ initialized, with doubly nested **parallel for** loops using $\Theta(n^2)$ work and $\Theta(\lg n)$ span.

3. Using the submatrices from step 1 and the matrices $S_1, S_2, \ldots, S_{10}$ created in step 2, recursively spawn computations of each of the seven $n/2 \times n/2$ matrix products $P_1, P_2, \ldots, P_7$, taking $7T_1(n/2)$ work and $T_\infty(n/2)$ span.
4. Update the four submatrices $C_{11}, C_{12}, C_{21}, C_{22}$ of the result matrix $C$ by adding or subtracting various $P_i$ matrices. Using doubly nested **parallel for** loops, computing all four submatrices takes $\Theta(n^2)$ work and $\Theta(\lg n)$ span.

Let's analyze this algorithm. Since the serial projection is the same as the original serial algorithm, the work is just the running time of the serial projection, namely, $\Theta(n^{\lg 7})$. As we did with P-MATRIX-MULTIPLY-RECURSIVE, we can devise a recurrence for the span. In this case, seven recursive calls execute in parallel, but since they all operate on matrices of the same size, we obtain the same recurrence (26.6) as we did for P-MATRIX-MULTIPLY-RECURSIVE, with solution $\Theta(\lg^2 n)$. Thus the parallel version of Strassen's method has parallelism $\Theta(n^{\lg 7}/\lg^2 n)$, which is large. Although the parallelism is slightly less than that of P-MATRIX-MULTIPLY-RECURSIVE, that's just because the work is also less.

## Exercises

### *26.2-1*

Draw the trace for computing P-MATRIX-MULTIPLY on $2 \times 2$ matrices, labeling how the vertices in your diagram correspond to strands in the execution of the algorithm. Assuming that each strand executes in unit time, analyze the work, span, and parallelism of this computation.

### *26.2-2*

Repeat Exercise 26.2-1 for P-MATRIX-MULTIPLY-RECURSIVE.

### *26.2-3*

Give pseudocode for a parallel algorithm that multiplies two $n \times n$ matrices with work $\Theta(n^3)$ but span only $\Theta(\lg n)$. Analyze your algorithm.

### *26.2-4*

Give pseudocode for an efficient parallel algorithm that multiplies a $p \times q$ matrix by a $q \times r$ matrix. Your algorithm should be highly parallel even if any of $p$, $q$, and $r$ equal 1. Analyze your algorithm.

***26.2-5***
Give pseudocode for an efficient parallel version of the Floyd-Warshall algorithm (see Section 23.2), which computes shortest paths between all pairs of vertices in an edge-weighted graph. Analyze your algorithm.

## 26.3 Parallel merge sort

We first saw serial merge sort in Section 2.3.1, and in Section 2.3.2 we analyzed its running time and showed it to be $\Theta(n \lg n)$. Because merge sort already uses the divide-and-conquer method, it seems like a terrific candidate for implementing using fork-join parallelism.

The procedure P-MERGE-SORT modifies merge sort to spawn the first recursive call. Like its serial counterpart MERGE-SORT on page 39, the P-MERGE-SORT procedure sorts the subarray $A[p:r]$. After the **sync** statement in line 8 ensures that the two recursive spawns in lines 5 and 7 have finished, P-MERGE-SORT calls the P-MERGE procedure, a parallel merging algorithm, which is on page 779, but you don't need to bother looking at it right now.

```
P-MERGE-SORT(A, p, r)
 1  if p ≥ r                    // zero or one element?
 2      return
 3  q = ⌊(p + r)/2⌋             // midpoint of A[p : r]
 4  // Recursively sort A[p : q] in parallel.
 5  spawn P-MERGE-SORT(A, p, q)
 6  // Recursively sort A[q + 1 : r] in parallel.
 7  spawn P-MERGE-SORT(A, q + 1, r)
 8  sync                        // wait for spawns
 9  // Merge A[p : q] and A[q + 1 : r] into A[p : r].
10  P-MERGE(A, p, q, r)
```

First, let's use work/span analysis to get some intuition for why we need a parallel merge procedure. After all, it may seem as though there should be plenty of parallelism just by parallelizing MERGE-SORT without worrying about parallelizing the merge. But what would happen if the call to P-MERGE in line 10 of P-MERGE-SORT were replaced by a call to the serial MERGE procedure on page 36? Let's call the pseudocode so modified P-NAIVE-MERGE-SORT.

Let $T_1(n)$ be the (worst-case) work of P-NAIVE-MERGE-SORT on an $n$-element subarray, where $n = r - p + 1$ is the number of elements in $A[p:r]$, and let $T_\infty(n)$

be the span. Because MERGE is serial with running time $\Theta(n)$, both its work and span are $\Theta(n)$. Since the serial projection of P-NAIVE-MERGE-SORT is exactly MERGE-SORT, its work is $T_1(n) = \Theta(n \lg n)$. The two recursive calls in lines 5 and 7 run in parallel, and so its span is given by the recurrence

$$\begin{aligned} T_\infty(n) &= T_\infty(n/2) + \Theta(n) \\ &= \Theta(n) , \end{aligned}$$

by case 1 of the master theorem. Thus the parallelism of P-NAIVE-MERGE-SORT is $T_1(n)/T_\infty(n) = \Theta(\lg n)$, which is an unimpressive amount of parallelism. To sort a million elements, for example, since $\lg 10^6 \approx 20$, it might achieve linear speedup on a few processors, but it would not scale up to dozens of processors.

The parallelism bottleneck in P-NAIVE-MERGE-SORT is plainly the MERGE procedure. If we asymptotically reduce the span of merging, the master theorem dictates that the span of parallel merge sort will also get smaller. When you look at the pseudocode for MERGE, it may seem that merging is inherently serial, but it's not. We can fashion a parallel merging algorithm. The goal is to reduce the span of parallel merging asymptotically, but if we want an efficient parallel algorithm, we must ensure that the $\Theta(n)$ bound on work doesn't increase.

Figure 26.6 depicts the divide-and-conquer strategy that we'll use in P-MERGE. The heart of the algorithm is a recursive auxiliary procedure P-MERGE-AUX that merges two sorted subarrays of an array $A$ into a subarray of another array $B$ in parallel. Specifically, P-MERGE-AUX merges $A[p_1 : r_1]$ and $A[p_2 : r_2]$ into subarray $B[p_3 : r_3]$, where $r_3 = p_3 + (r_1 - p_1 + 1) + (r_2 - p_2 + 1) - 1 = p_3 + (r_1 - p_1) + (r_2 - p_2) + 1$.

The key idea of the recursive merging algorithm in P-MERGE-AUX is to split each of the two sorted subarrays of $A$ around a pivot $x$, such that all the elements in the lower part of each subarray are at most $x$ and all the elements in the upper part of each subarray are at least $x$. The procedure can then recurse in parallel on two subtasks: merging the two lower parts, and merging the two upper parts. The trick is to find a pivot $x$ so that the recursion is not too lopsided. We don't want a situation such as that in QUICKSORT on page 183, where bad partitioning elements lead to a dramatic loss of asymptotic efficiency. We could opt to partition around a random element, as RANDOMIZED-QUICKSORT on page 192 does, but because the input subarrays are sorted, P-MERGE-AUX can quickly determine a pivot that always works well.

Specifically, the recursive merging algorithm picks the pivot $x$ as the middle element of the larger of the two input subarrays, which we can assume without loss of generality is $A[p_1 : r_1]$, since otherwise, the two subarrays can just switch roles. That is, $x = A[q_1]$, where $q_1 = \lfloor (p_1 + r_1)/2 \rfloor$. Because $A[p_1 : r_1]$ is sorted, $x$ is a median of the subarray elements: every element in $A[p_1 : q_1 - 1]$ is no more than $x$, and every element in $A[q_1 + 1 : r_1]$ is no less than $x$. Then the

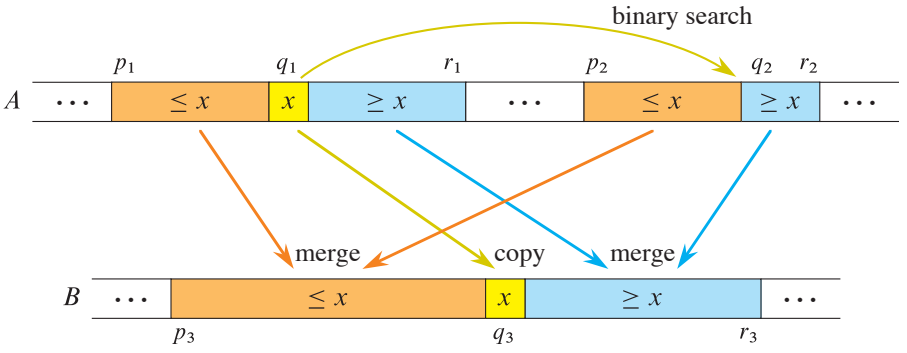


**Figure 26.6** The idea behind P-MERGE-AUX, which merges two sorted subarrays $A[p_1 : r_1]$ and $A[p_2 : r_2]$ into the subarray $B[p_3 : r_3]$ in parallel. Letting $x = A[q_1]$ (shown in yellow) be a median of $A[p_1 : r_1]$ and $q_2$ be a place in $A[p_2 : r_2]$ such that $x$ would fall between $A[q_2 - 1]$ and $A[q_2]$, every element in the subarrays $A[p_1 : q_1 - 1]$ and $A[p_2 : q_2 - 1]$ (shown in orange) is at most $x$, and every element in the subarrays $A[q_1 + 1 : r_1]$ and $A[q_2 + 1 : r_2]$ (shown in blue) is at least $x$. To merge, compute the index $q_3$ where $x$ belongs in $B[p_3 : r_3]$, copy $x$ into $B[q_3]$, and then recursively merge $A[p_1 : q_1 - 1]$ with $A[p_2 : q_2 - 1]$ into $B[p_3 : q_3 - 1]$ and $A[q_1 + 1 : r_1]$ with $A[q_2 : r_2]$ into $B[q_3 + 1 : r_3]$.

algorithm finds the "split point" $q_2$ in the smaller subarray $A[p_2 : r_2]$ such that all the elements in $A[p_2 : q_2 - 1]$ (if any) are at most $x$ and all the elements in $A[q_2 : r_2]$ (if any) are at least $x$. Intuitively, the subarray $A[p_2 : r_2]$ would still be sorted if $x$ were inserted between $A[q_2 - 1]$ and $A[q_2]$ (although the algorithm doesn't do that). Since $A[p_2 : r_2]$ is sorted, a minor variant of binary search (see Exercise 2.3-6) with $x$ as the search key can find the split point $q_2$ in $\Theta(\lg n)$ time in the worst case. As we'll see when we get to the analysis, even if $x$ splits $A[p_2 : r_2]$ badly—$x$ is either smaller than all the subarray elements or larger—we'll still have at least $1/4$ of the elements in each of the two recursive merges. Thus the larger of the recursive merges operates on at most $3/4$ elements, and the recursion is guaranteed to bottom out after $\Theta(\lg n)$ recursive calls.

Now let's put these ideas into pseudocode. We start with the serial procedure FIND-SPLIT-POINT$(A, p, r, x)$ on the next page, which takes as input a sorted subarray $A[p : r]$ and a key $x$. The procedure returns a split point of $A[p : r]$: an index $q$ in the range $p \le q \le r + 1$ such that all the elements in $A[p : q - 1]$ (if any) are at most $x$ and all the elements in $A[q : r]$ (if any) are at least $x$.

The FIND-SPLIT-POINT procedure uses binary search to find the split point. Lines 1 and 2 establish the range of indices for the search. Each time through the **while** loop, line 5 compares the middle element of the range with the search key $x$, and lines 6 and 7 narrow the search range to either the lower half or the upper half of the subarray, depending on the result of the test. In the end, after the range has been narrowed to a single index, line 8 returns that index as the split point.

```
FIND-SPLIT-POINT(A, p, r, x)
1  low = p                          // low end of search range
2  high = r + 1                     // high end of search range
3  while low < high                 // more than one element?
4      mid = ⌊(low + high)/2⌋       // midpoint of range
5      if x ≤ A[mid]                // is answer q ≤ mid?
6          high = mid               // narrow search to A[low : mid]
7      else low = mid + 1           // narrow search to A[mid + 1 : high]
8  return low
```

Because FIND-SPLIT-POINT contains no parallelism, its span is just its serial running time, which is also its work. On a subarray $A[p:r]$ of size $n = r - p + 1$, each iteration of the **while** loop halves the search range, which means that the loop terminates after $\Theta(\lg n)$ iterations. Since each iteration takes constant time, the algorithm runs in $\Theta(\lg n)$ (worst-case) time. Thus the procedure has work and span $\Theta(\lg n)$.

Let's now look at the pseudocode for the parallel merging procedure P-MERGE on the next page. Most of the pseudocode is devoted to the recursive procedure P-MERGE-AUX. The procedure P-MERGE itself is just a "wrapper" that sets up for P-MERGE-AUX. It allocates a new array $B[p:r]$ to hold the output of P-MERGE-AUX in line 1. It then calls P-MERGE-AUX in line 2, passing the indices of the two subarrays to be merged and providing $B$ as the output destination of the merged result, starting at index $p$. After P-MERGE-AUX returns, lines 3–4 perform a parallel copy of the output $B[p:r]$ into the subarray $A[p:r]$, which is where P-MERGE-SORT expects it.

The P-MERGE-AUX procedure is the interesting part of the algorithm. Let's start by understanding the parameters of this recursive parallel procedure. The input array $A$ and the four indices $p_1, r_1, p_2, r_2$ specify the subarrays $A[p_1:r_1]$ and $A[p_2:r_2]$ to be merged. The array $B$ and the index $p_3$ indicate that the merged result should be stored into $B[p_3:r_3]$, where $r_3 = p_3 + (r_1 - p_1) + (r_2 - p_2) + 1$, as we saw earlier. The end index $r_3$ of the output subarray is not needed by the pseudocode, but it helps conceptually to name the end index, as in the comment in line 13.

The procedure begins by checking the base case of the recursion and doing some bookkeeping to simplify the rest of the pseudocode. Lines 1 and 2 test whether the two subarrays are both empty, in which case the procedure returns. Line 3 checks whether the first subarray contains fewer elements than the second subarray. Since the number of elements in the first subarray is $r_1 - p_1 + 1$ and the number in the second subarray is $r_2 - p_2 + 1$, the test omits the two "+1's." If the first subarray